hello , i will make a snake game in mips assemplly give me pseudocode

Here's pseudocode for a MIPS snake game, organized so each piece maps cleanly to assembly routines.

Data layout

```
.data
GRID_W      = 32            # grid width in cells
GRID_H      = 16            # grid height in cells
snake_x[]   : array of words (max length = GRID_W * GRID_H)
snake_y[]   : array of words
snake_len   : word          # current length
dir_x, dir_y: words         # current direction (-1, 0, or 1)
food_x, food_y : words
score       : word
game_over   : word          # 0 = running, 1 = dead
```

If you're using the **MARS bitmap display** (recommended), set unit width/height (e.g. 16x16), display size (e.g. 512x256), and base address `0x10010000` (static data) or `0x10008000` (gp). Each cell is one word storing a color.

Main structure

```
main:
    call init_game
game_loop:
    call read_input
    call update_snake
    if game_over == 1: goto end_game
    call draw_screen
    call delay
    goto game_loop
end_game:
    print "Game Over", score
    exit
```

init_game

```
init_game:
    snake_len = 3
    snake_x[0] = 10; snake_y[0] = 8     # head
    snake_x[1] = 9;  snake_y[1] = 8
    snake_x[2] = 8;  snake_y[2] = 8     # tail
    dir_x = 1; dir_y = 0                # moving right
    score = 0
    game_over = 0
    call place_food
    return
```

read_input (non-blocking keyboard)

MARS uses memory-mapped keyboard: control at `0xFFFF0000`, data at `0xFFFF0004`.

```
read_input:
    ctrl = load word from 0xFFFF0000
    if (ctrl & 1) == 0: return          # no key pressed
    key = load word from 0xFFFF0004

    if key == 'w' and dir_y != 1:   dir_x = 0;  dir_y = -1
    if key == 's' and dir_y != -1:  dir_x = 0;  dir_y = 1
    if key == 'a' and dir_x != 1:   dir_x = -1; dir_y = 0
    if key == 'd' and dir_x != -1:  dir_x = 1;  dir_y = 0
    return
```

The `!=` checks prevent the snake from reversing directly into itself.

update_snake

```
update_snake:
    new_x = snake_x[0] + dir_x
    new_y = snake_y[0] + dir_y

    # wall collision
    if new_x < 0 or new_x >= GRID_W: game_over = 1; return
    if new_y < 0 or new_y >= GRID_H: game_over = 1; return

    # self collision
    for i = 0 to snake_len - 1:
        if snake_x[i] == new_x and snake_y[i] == new_y:
            game_over = 1
            return

    # check food
    ate = (new_x == food_x and new_y == food_y)

    if ate:
        snake_len += 1          # tail stays, so snake grows
        score += 1

    # shift body back (start from the end)
    for i = snake_len - 1 down to 1:
        snake_x[i] = snake_x[i-1]
        snake_y[i] = snake_y[i-1]

    snake_x[0] = new_x
    snake_y[0] = new_y

    if ate: call place_food
    return
```

Note: when growing, increment `snake_len` *before* shifting so the old tail position is preserved in the new last slot.

place_food

```
place_food:
retry:
    food_x = random(0, GRID_W)      # syscall 42 (random int range)
    food_y = random(0, GRID_H)
    for i = 0 to snake_len - 1:
        if snake_x[i] == food_x and snake_y[i] == food_y:
            goto retry              # don't spawn on snake
    return
```

draw_screen

```
draw_screen:
    clear display (fill all cells with BLACK)

    # draw food
    set_pixel(food_x, food_y, RED)

    # draw snake
    set_pixel(snake_x[0], snake_y[0], YELLOW)   # head
    for i = 1 to snake_len - 1:
        set_pixel(snake_x[i], snake_y[i], GREEN)
    return

set_pixel(x, y, color):
    offset = (y * GRID_W + x) * 4
    store color at BASE_ADDR + offset
```

For efficiency, instead of clearing the whole screen each frame, you can just erase the old tail and draw the new head.

delay

```
delay:
    syscall 32 (sleep) with ~100–150 ms
    return
```

Helpful MARS syscalls

Purpose	Syscall	Notes
Sleep	32	`$a0` = milliseconds
Random int range	42	`$a0` = id, `$a1` = upper bound, result in `$a0`
Print string	4	`$a0` = address
Print int	1	`$a0` = value
Exit	10	

Suggested build order

1. Draw a single pixel at a fixed position.
2. Move it with keyboard input.
3. Add the snake as an array and the shifting logic.
4. Add food and growth.
5. Add wall and self collision.
6. Add score and polish.

If you want, I can write the actual MIPS code for any section, or the whole thing, targeting MARS. Just tell me which simulator you're using.



now i will start really , give me the steps , and i will use these config

Revising the earlier pseudocode for the new grid size ▸

Your settings give a **64 x 32 grid** (512/8 by 256/8), so use `GRID_W = 64` and `GRID_H = 32` instead of the 32x16 from my pseudocode. Each cell is one 4-byte word, so the display uses 64 x 32 x 4 = **8192 bytes**.

Steps

Step 0: Set up MARS

1. In the Bitmap Display, click **Connect to MIPS**.
2. Open *Tools > Keyboard and Display MMIO Simulator* and click **Connect to MIPS** there too.
3. Before running, click in the *bottom* text box of the keyboard tool. Your W/A/S/D keys are typed there.

Step 1: Protect the display memory

Because the base address is `0x10010000 (static data)`, your `.data` section starts at the same address as the screen. Variables placed first would show up as colored pixels. Reserve the screen area first:

asm

```
.data
display:   .space 8192        # 64*32*4 bytes, this IS the screen
snake_x:   .space 8192        # max 2048 segments * 4 bytes
snake_y:   .space 8192
snake_len: .word 3
dir_x:     .word 1
dir_y:     .word 0
food_x:    .word 0
food_y:    .word 0
score:     .word 0
```

Step 2: Write `set_pixel(x, y, color)` and test it

Formula: `address = 0x10010000 + (y * 64 + x) * 4`

asm

```
# $a0 = x, $a1 = y, $a2 = color
set_pixel:
    sll  $t0, $a1, 6          # y * 64
    add  $t0, $t0, $a0        # + x
    sll  $t0, $t0, 2          # * 4
    lui  $t1, 0x1001          # base 0x10010000
    add  $t0, $t0, $t1
    sw   $a2, 0($t0)
    jr   $ra
```

Test it by drawing one pixel (for example `0x00FF0000` for red) in `main`. Don't move on until it appears.

Step 3: Write `clear_screen`

Loop 2048 times, storing `0` (black) at each word starting from `0x10010000`.

Step 4: Draw the initial snake and food

Initialize `snake_x/snake_y` with 3 segments, then loop over them calling `set_pixel`. Draw a fixed food pixel in red.

Step 5: Game loop with delay

Structure: `read_input > update > draw > sleep`. Use syscall 32 with `$a0 = 100` (milliseconds). At this point just have it loop and redraw.

Step 6: Keyboard input

Check `0xFFFF0000` bit 0. If it is set, read the key from `0xFFFF0004` and compare with ASCII `'w'`=119, `'a'`=97, `'s'`=115, `'d'`=100. Update `dir_x/dir_y`, ignoring direct reversals.

Step 7: Move the snake

Compute the new head position, shift the body (loop from the end down to index 1), and store the new head at index 0. Erase the old tail with black and draw the new head. The snake should now move around the screen.

Step 8: Collisions

- Walls: exit if `x < 0`, `x >= 64`, `y < 0` or `y >= 32`.
- Self: loop through the body and compare each segment with the new head.

Step 9: Food and growth

If the new head equals the food position, increment `snake_len` and `score` and call `place_food`. Use syscall 42 with `$a1 = 64` for x and `$a1 = 32` for y, and retry if the food lands on the snake.

Step 10: Game over and polish

Print the score with syscalls 4 and 1, then exit with syscall 10. Optional extras: speed up as the score grows, use a different head color, add a restart option.

Tips

- Use `$s` registers for values that must survive function calls, and save `$ra` on the stack in any function that calls another.
- Build and test one step at a time. Debugging assembly is much harder when many untested pieces are combined.

Send me your code after any step if something doesn't work, or tell me which step you'd like me to write in full.

give me full code

Created a file, shared a file ▸

I've written the full game in `snake.asm`, set up for your 8x8 units, 512x256 display and `0x10010000` base address (a 64x32 grid). I haven't run it in MARS.

To run it:

1. Open the file in MARS and assemble it.
2. Open *Tools* > *Bitmap Display* and click **Connect to MIPS**.
3. Open *Tools* > *Keyboard and Display MMIO Simulator* and click **Connect to MIPS**.
4. Run the program, then click the **bottom** text box of the keyboard tool. Steer with W, A, S, D.

How it plays:

- The head is yellow, the body green and the food red.
- The snake dies when it hits a wall or itself, then the score is printed in the Run I/O console.
- You can't reverse straight into yourself.
- Speed is the `li $a0, 100` line in `game_loop`. Lower it for a faster game.

If something doesn't work, tell me what you see (blank screen, no movement, a crash) and I'll help you fix it.

`set_pixel` draws one colored cell on the bitmap display. It takes three inputs:

- `$a0` = x (column, 0 to 63)
- `$a1` = y (row, 0 to 31)
- `$a2` = color, as `0x00RRGGBB`

asm

```
set_pixel:
    sll  $t0, $a1, 6      # y * 64   (shift left 6 = multiply by 64)
    add  $t0, $t0, $a0    # + x      -> cell number in the grid
    sll  $t0, $t0, 2      # * 4      -> each cell is one 4-byte word
    lui  $t1, 0x1001      # $t1 = 0x10010000, the display base address
    add  $t0, $t0, $t1    # address = base + offset
    sw   $a2, 0($t0)      # store the color; the display shows it
    jr   $ra
```

The display is a grid stored row by row in memory, one word per cell. So the cell at (x, y) is at:

```
address = 0x10010000 + (y * 64 + x) * 4
```

The 64 is the grid width (512 / 8). For example, (x=3, y=2) gives (2*64 + 3) * 4 = 524 bytes past the base.

Example usage: draw a red pixel at x=10, y=5:

asm

```
li   $a0, 10
li   $a1, 5
li   $a2, 0x00FF0000    # red
jal  set_pixel
```

Some common colors:

Color	Value
Black (erase)	`0x00000000`
Red	`0x00FF0000`
Green	`0x0000FF00`
Blue	`0x000000FF`
Yellow	`0x00FFFF00`
White	`0x00FFFFFF`

The function only uses `$t0` and `$t1`, so it won't disturb your `$s` registers. It also doesn't call anything, so it doesn't need to save `$ra`.

build starting snake (head at 10,16 moving right) 10 means row ?

No, **10 is the column (x)** and **16 is the row (y)**. The head is at (x=10, y=16).

- **x = 10** means 10 cells from the left edge (columns run 0 to 63).
- **y = 16** means 16 cells down from the top edge (rows run 0 to 31, with row 0 at the top).

Since the grid is 32 rows tall, row 16 is about the vertical middle of the screen.

The snake is built horizontally, all on row 16:

```
x:  8    9    10
    tail body head   -> moving right (dir_x = 1, dir_y = 0)
y:  16   16   16
```

In the code, `snake_x` holds the columns `10, 9, 8` and `snake_y` holds the rows `16, 16, 16`. Index 0 is always the head, so the head is (10, 16), the body (9, 16) and the tail (8, 16).

Moving right adds 1 to x, so the head goes to (11, 16), then (12, 16), and so on. Moving up subtracts 1 from y (row 0 is the top), which is why `'w'` sets `dir_y = -1`.

```
la  $t0, snake_x
la  $t1, snake_y
li  $t2, 10
```

```
sw  $t2, 0($t0)
li  $t2, 9
sw  $t2, 4($t0)
li  $t2, 8
sw  $t2, 8($t0)
li  $t2, 16
sw  $t2, 0($t1)
sw  $t2, 4($t1)
sw  $t2, 8($t1)?
```

This block stores the starting positions of the 3 snake segments into the two arrays in memory.

asm

```
la   $t0, snake_x      # $t0 = address of the snake_x array
la   $t1, snake_y      # $t1 = address of the snake_y array
```

`la` (load address) puts the *address* of each array into a register, so we can write into it.

The x values (columns):

asm

```
li   $t2, 10           # $t2 = 10
sw   $t2, 0($t0)       # snake_x[0] = 10   (head)
li   $t2, 9
sw   $t2, 4($t0)       # snake_x[1] = 9    (body)
li   $t2, 8
sw   $t2, 8($t0)       # snake_x[2] = 8    (tail)
```

The y values (rows):

asm

```
li   $t2, 16           # $t2 = 16
sw   $t2, 0($t1)       # snake_y[0] = 16
sw   $t2, 4($t1)       # snake_y[1] = 16
sw   $t2, 8($t1)       # snake_y[2] = 16
```

All three segments are on row 16, so the same register value is stored three times without reloading it.

Why the offsets are 0, 4, 8: each element is a word (4 bytes), so element *i* is at byte offset `i * 4`.

Element	Offset	x	y
0 (head)	0	10	16
1 (body)	4	9	16
2 (tail)	8	8	16

After this runs, memory holds:

```
snake_x: [10, 9, 8, ...]
snake_y: [16, 16, 16, ...]
```

Segment *i* is the pair `(snake_x[i], snake_y[i])`, so the segments are (10,16), (9,16) and (8,16).

Note that this only stores the data. Nothing appears on screen until `set_pixel` is called for each segment, which is what the next lines in `init_game` do.

```
li  $t2, 3
sw  $t2, snake_len
li  $t2, 1
sw  $t2, dir_x
sw  $zero, dir_y
sw  $zero, score
sw  $zero, game_over but we initilize the se vars in data segment
```

Weighing why variables get set again in code.

You're right, those lines are redundant for a single run. The `.data` section already sets the same values:

asm

```
snake_len:  .word 3
dir_x:      .word 1
dir_y:      .word 0
score:      .word 0
game_over:  .word 0
```

`.data` values are loaded once, when the program is assembled and started. Setting them again in `init_game` changes nothing the first time, so you can delete those lines and the game works the same.

I put them in `init_game` for **restarting**. If you later add a "play again" option that calls `init_game` a second time, `.data` won't reload on its own. The variables would still hold the old values: the old length, the old score and `game_over = 1`. Resetting them in code makes `init_game` start a fresh game every time it runs.

So you have two options:

- **No restart planned:** delete those lines. Keep the `.data` initial values.
- **Restart planned:** keep the lines in `init_game`. The `.data` values then become unnecessary, though harmless.

The two arrays are different. `snake_x` and `snake_y` are declared with `.space`, which only reserves memory (zeros), so their starting positions (10, 9, 8 and 16) still have to be stored by code, as in the block you asked about before.

li $a2, 0x00FFFF00 but immediate size is 2 bytes

You're right that a real MIPS instruction can only hold a 16-bit immediate, but `li` isn't a real instruction. It's a **pseudo-instruction**, and MARS expands it into real ones.

For a value that doesn't fit in 16 bits, `li $a2, 0x00FFFF00` becomes:

asm

```
lui  $a2, 0x00FF       # load upper 16 bits: $a2 = 0x00FF0000
ori  $a2, $a2, 0xFF00  # OR in the lower 16 bits: $a2 = 0x00FFFF00
```

- `lui` (load upper immediate) puts a 16-bit value in the **top half** of the register and zeros the bottom half.
- `ori` fills in the **bottom half**.

You can see the expansion in MARS. After assembling, the **Text Segment** view shows the "Basic" column with the real instructions next to your source line.

If a value fits in 16 bits, `li` expands to a single instruction. For example, `li $t2, 10` becomes `addiu $t2, $zero, 10`.

`la` and `lw $t0, dir_x` work the same way. They are expanded with `lui` plus a lower-half instruction (`ori` for `la`, `lw` with an offset for loads), and sometimes the assembler register `$at`.

You can write the two instructions yourself if you prefer. The result is identical.

```
read_input:
  lui  $t0, 0xFFFF
  lw   $t1, 0($t0)          # receiver control
  andi $t1, $t1, 1
  beqz $t1, ri_done         # no key waiting
  lw   $t2, 4($t0)          # receiver data (ASCII)
  lw   $t3, dir_x
  lw   $t4, dir_y?
```

This part of `read_input` checks whether a key was pressed. If one was, it reads the key and loads the current direction.

Getting the keyboard address:

```asm
lui  $t0, 0xFFFF        # $t0 = 0xFFFF0000
```

MARS maps the keyboard into memory. Reading from these addresses talks to the Keyboard and Display MMIO Simulator, not to normal RAM. `lui` puts `0xFFFF` in the top half, so `$t0` holds the base address `0xFFFF0000`.

Checking if a key is waiting:

```asm
lw   $t1, 0($t0)        # read the control register at 0xFFFF0000
andi $t1, $t1, 1        # keep only bit 0
beqz $t1, ri_done       # bit 0 == 0 -> no key, leave the function
```

Bit 0 of the control register is the "ready" flag. It is 1 when a key has been typed and not yet read, and 0 otherwise. `andi` with 1 masks out every other bit, so `$t1` becomes exactly 0 or 1. If it's 0, the code jumps to `ri_done` and the snake just keeps its direction. This is what makes the input **non-blocking**: the game never waits for you to press something.

Reading the key:

```asm
lw   $t2, 4($t0)        # read the data register at 0xFFFF0004
```

The data register holds the ASCII code of the key (`'w'` = 119, `'a'` = 97, `'s'` = 115, `'d'` = 100). Reading it also clears the ready flag, so the same key isn't processed twice.

Loading the current direction:

asm

```
lw   $t3, dir_x        # $t3 = current dir_x
lw   $t4, dir_y        # $t4 = current dir_y
```

These are needed for the reverse check later. For example, if the snake is moving right (`dir_x = 1`) and you press `'a'`, the code compares against `$t3` and ignores the key, so the snake can't turn back into itself.

Register	Holds
`$t0`	`0xFFFF0000` (keyboard base)
`$t1`	ready flag (0 or 1)
`$t2`	ASCII code of the key pressed
`$t3`	current `dir_x`
`$t4`	current `dir_y`

andi $t1, $t1, 1 but here the read data may the bit 0 be either 0 or 1

Checking which register bit signals readiness.
Yes, bit 0 can be either 0 or 1, and that is exactly what the code tests. The `andi` is what turns that bit into a clean answer.

What gets read here is the control register, not the key.

asm

```
lw   $t1, 0($t0)       # control register at 0xFFFF0000
```

This is a 32-bit word, but only bit 0 means anything:

- bit 0 = 1 means a key has been typed and is waiting to be read
- bit 0 = 0 means no key is waiting

The other 31 bits aren't relied on, and they may hold anything.

What `andi` does:

asm

```
andi $t1, $t1, 1       # $t1 = $t1 AND 0b000...0001
```

ANDing with 1 clears bits 1 to 31 and keeps only bit 0, so `$t1` becomes exactly 0 or 1:

```
control = ...xxxx1  ->  AND 1  ->  1   (key waiting)
control = ...xxxx0  ->  AND 1  ->  0   (no key)
```

Then the branch uses that result:

asm

```
beqz $t1, ri_done      # if 0 -> no key, skip reading
```

- result 0: jump to `ri_done`, so the game continues with no input.
- result 1: fall through and read the key from `0xFFFF0004`.

Without the `andi`, `beqz` would test the whole word, and a nonzero value in some other bit could make it wrongly think a key is waiting. Masking makes the test depend only on bit 0.

The ASCII code of the key is not in this register. It comes from the second one, `lw $t2, 4($t0)`, and that read is only done once the ready bit says there is something to read.

```
li  $t5, 119          # 'w' -> up
  bne  $t2, $t5, ri_s
  li  $t6, 1
  beq  $t4, $t6, ri_done      # currently moving down
  sw  $zero, dir_x
  li  $t6, -1
  sw  $t6, dir_y
  j  ri_done?
```

This block handles the **'w' (up)** key.

asm

```
li   $t5, 119                  # $t5 = ASCII code of 'w'
bne  $t2, $t5, ri_s            # key is NOT 'w' -> jump to the 's' check
```

`$t2` holds the key that was pressed. If it isn't `'w'`, the code skips to the next check (`ri_s`).

asm

```
li   $t6, 1
beq  $t4, $t6, ri_done          # currently moving down
```

The key is `'w'`, so first the reverse check. `$t4` is the current `dir_y`. If `dir_y == 1`, the snake is moving **down**, and turning straight up would send the head into its own body. So the code jumps to `ri_done` and ignores the key.

asm

```
sw   $zero, dir_x               # dir_x = 0
li   $t6, -1
sw   $t6, dir_y                 # dir_y = -1
```

The move is allowed, so the direction is set to up: no horizontal movement and `dir_y = -1`. Row 0 is the top of the screen, so going up means the row number gets smaller.

asm

```
j    ri_done
```

Done. The code jumps over the `'s'`, `'a'` and `'d'` checks so they don't run for this key.

The decision in table form:

Situation	Result
key is not `'w'`	go test `'s'`
key is `'w'` and snake moving down (`dir_y = 1`)	ignored
key is `'w'` otherwise	`dir_x = 0`, `dir_y = -1`

Pressing `'w'` while already moving up just sets the same direction again, which is harmless. The other three keys follow the same pattern with different values:

Key	Ignored if moving	New `dir_x`	New `dir_y`
`w`	down (`dir_y = 1`)	0	-1
`s`	up (`dir_y = -1`)	0	1
`a`	right (`dir_x = 1`)	-1	0
`d`	left (`dir_x = -1`)	1	0

dir_y=1 then it is moving sown ?

Yes. `dir_y` is added to the head's y (the row) each move, and row 0 is at the **top** of the screen, so rows get bigger going down:

```
row 0   <- top of screen
row 1
 ...
row 31  <- bottom of screen
```

- `dir_y = 1` means y increases by 1 each step, so the snake moves **down**.
- `dir_y = -1` means y decreases by 1 each step, so the snake moves **up**.

For example, a head at (10, 16) with `dir_y = 1` goes to (10, 17), then (10, 18), which is one row lower each time.

This is the reverse of a math graph, where y increases upward. On screens, and in the bitmap display, y counts from the top.

That's why `'w'` sets `dir_y = -1` and `'s'` sets `dir_y = 1`, and why the check on `'w'` is "ignore if `dir_y == 1`": that means the snake is moving down, and going straight up would reverse it into its own body.

```
addi $sp, $sp, -4
  sw  $ra, 0($sp)

  la  $s4, snake_x
  la  $s5, snake_y
  lw  $s0, 0($s4)          # head x
  lw  $s1, 0($s5)          # head y
  lw  $t0, dir_x
  lw  $t1, dir_y
  add $s2, $s0, $t0        # new x
  add $s3, $s1, $t1        # new y
```

This is the start of `update_snake`. It saves the return address, then works out where the head will be after this move.

Saving the return address:

```asm
addi $sp, $sp, -4      # make room on the stack (grow it by 4 bytes)
sw   $ra, 0($sp)       # save $ra there
```

`update_snake` calls `set_pixel` and `place_food` with `jal`, and every `jal` overwrites `$ra`. Without saving it first, the final `jr $ra` would jump to the wrong place. The function restores it at the end:

```asm
lw   $ra, 0($sp)
addi $sp, $sp, 4
```

Getting the array addresses:

```asm
la   $s4, snake_x      # $s4 = address of snake_x
la   $s5, snake_y      # $s5 = address of snake_y
```

These are kept in `$s` registers because they're used many times later, and `$s` registers survive the `jal` calls.

Reading the current head:

```asm
lw   $s0, 0($s4)       # $s0 = snake_x[0] = head x
lw   $s1, 0($s5)       # $s1 = snake_y[0] = head y
```

Index 0 is the head, so the offset is 0.

Reading the direction:

```asm
lw   $t0, dir_x        # -1, 0 or 1
lw   $t1, dir_y        # -1, 0 or 1
```

Computing the new head:

```asm
```

```
add  $s2, $s0, $t0     # new x = head x + dir_x
add  $s3, $s1, $t1     # new y = head y + dir_y
```

Nothing is stored yet. The new position is only computed, so the next code can check whether the move is legal (walls, self, food) before the snake actually moves.

Example: the head is at (10, 16) and the snake is moving right (`dir_x = 1` , `dir_y = 0`):

Register	Value
`$s0` , `$s1`	10, 16 (current head)
`$s2` , `$s3`	11, 16 (new head)

```
# ---- self collision ----
  lw  $s6, snake_len
  li  $t0, 0
us_self:
  bge  $t0, $s6, us_selfok
  sll  $t1, $t0, 2
  add  $t2, $s4, $t1
  lw  $t3, 0($t2)
  bne  $t3, $s2, us_next
  add  $t2, $s5, $t1
  lw  $t3, 0($t2)
  beq  $t3, $s3, us_dead
us_next:
  addi $t0, $t0, 1
  j  us_self
```

Show more

This loop checks whether the new head position lands on any segment of the snake's body. If it does, the snake has hit itself and the game ends.

Setup:

asm

```
lw   $s6, snake_len    # $s6 = number of segments
li   $t0, 0            # $t0 = i = 0 (loop counter)
```

Loop condition:

asm

```
us_self:
    bge  $t0, $s6, us_selfok   # if i >= snake_len -> checked everything, no colli
```

When `i` reaches the length, every segment has been checked without a hit, so the code jumps to `us_selfok` and the move continues.

Getting segment i:

```asm
sll  $t1, $t0, 2       # $t1 = i * 4  (byte offset)
add  $t2, $s4, $t1     # $t2 = address of snake_x[i]
lw   $t3, 0($t2)       # $t3 = snake_x[i]
```

Comparing x first:

```asm
bne  $t3, $s2, us_next # segment x != new head x -> can't be a hit, skip to next s
```

Both x and y must match for a collision. If x already differs, the y check is skipped, which saves work.

Comparing y:

```asm
add  $t2, $s5, $t1     # $t2 = address of snake_y[i]
lw   $t3, 0($t2)       # $t3 = snake_y[i]
beq  $t3, $s3, us_dead # y also matches -> collision, game over
```

Same offset `$t1`, since `snake_x` and `snake_y` are indexed together. If x and y both match, the code jumps to `us_dead`, which sets `game_over = 1`.

Next segment:

```asm
us_next:
    addi $t0, $t0, 1   # i++
    j    us_self       # repeat
```

In pseudocode:

```
for i = 0 to snake_len - 1:
    if snake_x[i] == new_x and snake_y[i] == new_y:
        game over
```

Example: the snake is (10,16), (9,16), (8,16) and the new head is (11,16).

i	segment	match?
0	(10,16)	x differs, skip
1	(9,16)	x differs, skip
2	(8,16)	x differs, skip

`i` reaches 3, which equals `snake_len`, so there is no collision.

One detail: the loop checks all segments, including the tail. Normally the tail moves away on the same step, so moving into the tail's current cell would actually be safe in classic snake. This version treats it as a collision, which is slightly stricter but simpler. If you want the classic behavior, loop only up to `snake_len - 1` (exclusive of the tail).

```
# ---- did we eat the food? ----
  lw  $t0, food_x
  lw  $t1, food_y
  li  $s7, 0
  bne $s2, $t0, us_noeat
  bne $s3, $t1, us_noeat
  li  $s7, 1 ?
```

This block checks whether the new head position is on the food. It sets a flag in `$s7`: 0 for no, 1 for yes.

Loading the food position:

asm

```
lw   $t0, food_x       # $t0 = food column
lw   $t1, food_y       # $t1 = food row
```

Assuming "not eaten" first:

asm

```
li   $s7, 0            # flag = 0 (default: didn't eat)
```

The flag starts at 0, so the only way it becomes 1 is if both coordinates match.

Comparing x and y:

```asm
bne  $s2, $t0, us_noeat    # new head x != food x -> not eaten, skip
bne  $s3, $t1, us_noeat    # new head y != food y -> not eaten, skip
```

Both must match for the snake to be on the food. If either one differs, the code jumps to `us_noeat` and the flag stays 0.

Both matched:

```asm
li   $s7, 1            # flag = 1 (ate the food)
us_noeat:              # execution continues from here either way
```

If the code gets past both `bne` instructions, the head is exactly on the food, so the flag is set to 1. The `us_noeat` label sits right after, so both paths continue from the same place.

In pseudocode:

```
ate = 0
if new_x == food_x and new_y == food_y:
    ate = 1
```

Why a flag is needed: the result is used in several places later in the function:

- If `ate == 0`: erase the tail, so the snake keeps its length.
- If `ate == 1`: keep the tail (the snake grows), increase `snake_len` and `score`, and call `place_food`.

`$s7` is used instead of a `$t` register because later code calls `set_pixel` with `jal`, and `$s` registers are the ones expected to hold their values across calls.

```
bnez $s7, us_grow
  addi $t0, $s6, -1
  sll  $t0, $t0, 2
  add  $t1, $s4, $t0
  lw   $a0, 0($t1)
```

```
add  $t1, $s5, $t0
lw   $a1, 0($t1)
li   $a2, 0          # black
jal  set_pixel
j    us_shift
```

This block erases the tail from the screen when the snake did **not** eat. Not growing means the snake moves forward by adding a new head and dropping the old tail.

Branching on the flag:

```asm
bnez $s7, us_grow      # ate food (flag = 1) -> skip erasing, go grow
```

If the snake ate, the tail must stay, so the code jumps to `us_grow` and this block is skipped. If the flag is 0, execution continues into the erase code.

Finding the tail:

```asm
addi $t0, $s6, -1      # $t0 = snake_len - 1  (index of the tail)
sll  $t0, $t0, 2       # $t0 = index * 4      (byte offset)
```

The last segment is at index `len - 1`. For a length of 3, that's index 2, which is offset 8.

Reading the tail's coordinates:

```asm
add  $t1, $s4, $t0     # $t1 = address of snake_x[len-1]
lw   $a0, 0($t1)       # $a0 = tail x   (argument 1 for set_pixel)
add  $t1, $s5, $t0     # $t1 = address of snake_y[len-1]
lw   $a1, 0($t1)       # $a1 = tail y   (argument 2 for set_pixel)
```

The values are loaded straight into `$a0` and `$a1` because `set_pixel` expects x and y there.

Drawing black over it:

```asm
li   $a2, 0            # color = black
jal  set_pixel         # paint the tail cell black (erase it)
```

Skipping the grow code:

```asm
j    us_shift          # jump over the us_grow code
```

Without this jump, execution would fall into `us_grow` and increase the length and score even though nothing was eaten.

In pseudocode:

```
if not ate:
    erase pixel at (snake_x[len-1], snake_y[len-1])
else:
    (grow instead)
```

Why the screen is updated this way: only two pixels change per move. One is the old tail, erased here. The other is the new head, which is drawn later in the function. That is much faster than clearing and redrawing the whole screen every frame.

Example: the snake is (10,16), (9,16), (8,16), with `len = 3` and no food eaten.

- Tail index = 2, offset = 8, so the tail is (8,16).
- Pixel (8,16) is painted black.
- The shift step then makes the array (11,16), (10,16), (9,16), and the screen matches.

```
us_shift:
  addi $t0, $s6, -1
us_shift_loop:
  blez $t0, us_shift_done
  sll  $t1, $t0, 2          # offset i
  addi $t2, $t1, -4         # offset i-1
  add  $t3, $s4, $t2
  lw   $t4, 0($t3)
  add  $t3, $s4, $t1
  sw   $t4, 0($t3)
  add  $t3, $s5, $t2
  lw   $t4, 0($t3)
  add  $t3, $s5, $t1
  sw   $t4, 0($t3)
  addi $t0, $t0, -1
  j    us_shift_loop
us_shift_done:
```

This loop moves every segment one slot toward the tail, which opens up index 0 for the new head. It copies from the back to the front: `pos[i] = pos[i-1]`.

Starting at the tail:

```asm
us_shift:
    addi $t0, $s6, -1      # $t0 = i = snake_len - 1 (index of the last segment)
```

When the snake ate, `$s6` was already increased in `us_grow`, so this index is the new last slot. The old tail's position is still sitting in `pos[len-2]`, and it gets copied into the new slot, which is how the snake grows.

Loop condition:

```asm
us_shift_loop:
    blez $t0, us_shift_done   # if i <= 0 -> stop
```

The loop runs for i = len-1 down to 1. It stops at 0 because index 0 is the head slot, which gets the new position afterwards.

Offsets:

```asm
sll  $t1, $t0, 2          # $t1 = i * 4        (offset of segment i)
addi $t2, $t1, -4         # $t2 = (i-1) * 4    (offset of segment i-1)
```

Copying x:

```asm
add  $t3, $s4, $t2        # address of snake_x[i-1]
lw   $t4, 0($t3)          # read snake_x[i-1]
add  $t3, $s4, $t1        # address of snake_x[i]
sw   $t4, 0($t3)          # snake_x[i] = snake_x[i-1]
```

Copying y:

```asm
add  $t3, $s5, $t2        # address of snake_y[i-1]
lw   $t4, 0($t3)          # read snake_y[i-1]
add  $t3, $s5, $t1        # address of snake_y[i]
sw   $t4, 0($t3)          # snake_y[i] = snake_y[i-1]
```

It's the same copy as for x, but on the `snake_y` array.

Next iteration:

```asm
addi $t0, $t0, -1          # i--
j    us_shift_loop
```

In pseudocode:

```
for i = len-1 down to 1:
    snake_x[i] = snake_x[i-1]
    snake_y[i] = snake_y[i-1]
```

Why it goes backward: if you copied from the front (`pos[1] = pos[0]`, then `pos[2] = pos[1]`), the first copy would overwrite `pos[1]`, and every later slot would end up with the head's value. Going from the back, each source slot is read before it is overwritten.

Example: the snake is (10,16), (9,16), (8,16), with `len = 3`.

Step	i	Action	Arrays after
start			x: [10, 9, 8]
1	2	x[2] = x[1]	x: [10, 9, 9]
2	1	x[1] = x[0]	x: [10, 10, 9]
stop	0	exit	

Slot 0 still holds the old head, and the code right after (`sw $s2, 0($s4)` and `sw $s3, 0($s5)`) overwrites it with the new head (11,16). The final result is x: [11, 10, 9], which is the snake moved one cell right.

```
pf_chk:
  bge  $t0, $t6, pf_ok
  sll  $t1, $t0, 2
  add  $t2, $t4, $t1
  lw   $t3, 0($t2)
  bne  $t3, $s0, pf_nx
  add  $t2, $t5, $t1
  lw   $t3, 0($t2)
  beq  $t3, $s1, pf_retry
```

Tracing how the snake's registers are set up for the overlap check ▸

This loop is in `place_food`. It checks whether the randomly chosen food position lands on the snake, and if it does, it picks a new random position.

What's already set before the loop:

Register	Holds
`$s0`, `$s1`	candidate food x, y (just generated randomly)
`$t4`, `$t5`	addresses of `snake_x` and `snake_y`
`$t6`	`snake_len`
`$t0`	`i`, the loop counter, starting at 0

Loop condition:

```asm
pf_chk:
    bge  $t0, $t6, pf_ok       # i >= snake_len -> checked every segment, positior
```

If the loop gets through all segments without a hit, the food position is valid and the code jumps to `pf_ok`, which saves and draws the food.

Reading segment i's x:

```asm
sll  $t1, $t0, 2              # byte offset = i * 4
add  $t2, $t4, $t1            # address of snake_x[i]
lw   $t3, 0($t2)              # $t3 = snake_x[i]
```

Comparing x:

```asm
bne  $t3, $s0, pf_nx          # x differs -> this segment can't match, go to next
```

Comparing y:

```asm
```

```
add  $t2, $t5, $t1            # address of snake_y[i]
lw   $t3, 0($t2)              # $t3 = snake_y[i]
beq  $t3, $s1, pf_retry       # y matches too -> food is ON the snake, pick a new
```

Only when both x and y match is the position bad. The jump to `pf_retry` goes back to the top of `place_food`, which generates a fresh random x and y and checks again.

Moving to the next segment (the two lines after the part you pasted):

asm

```
pf_nx:
    addi $t0, $t0, 1          # i++
    j    pf_chk
```

In pseudocode:

```
for i = 0 to snake_len - 1:
    if snake_x[i] == food_x and snake_y[i] == food_y:
        pick a new random position and start over
food position is valid
```

This is the same pattern as the self-collision check in `update_snake`, except that a hit here means "try again" rather than "game over".

Why it matters: the grid has 2048 cells and the snake takes up only a few, so a retry is rare early on. As the snake grows long, retries become more common, but the loop always ends eventually as long as at least one free cell exists.

```
sw  $s0, food_x
  sw  $s1, food_y
  move $a0, $s0
  move $a1, $s1
  li  $a2, 0x00FF0000       # red
  jal set_pixel

  lw  $ra, 0($sp)
  addi $sp, $sp, 4
  jr  $ra
```

This is the end of `place_food` (the `pf_ok` section). The position is now known to be free, so the code saves it, draws the food and returns.

Saving the food position:

```asm
sw   $s0, food_x       # food_x = chosen x
sw   $s1, food_y       # food_y = chosen y
```

The position is stored in memory so `update_snake` can compare the new head against it later (the "did we eat the food?" check).

Drawing the food:

```asm
move $a0, $s0          # argument 1: x
move $a1, $s1          # argument 2: y
li   $a2, 0x00FF0000   # argument 3: red
jal  set_pixel         # draw a red pixel at (x, y)
```

`set_pixel` expects its inputs in `$a0`, `$a1` and `$a2`, so the values are copied there first. `move $a0, $s0` is a pseudo-instruction meaning "copy `$s0` into `$a0`".

Returning to the caller:

```asm
lw   $ra, 0($sp)       # restore the saved return address
addi $sp, $sp, 4       # give the stack space back
jr   $ra               # jump back to whoever called place_food
```

At the top of `place_food` the code did:

```asm
addi $sp, $sp, -4
sw   $ra, 0($sp)
```

That was needed because `place_food` calls other code with `jal` (the `syscall`s don't change `$ra`, but the `jal set_pixel` does). The `jal` overwrites `$ra`, so without restoring the saved value, `jr $ra` would jump back into `place_food` itself and loop forever. Every function that uses `jal` follows this pattern: save `$ra` on entry, restore it before `jr $ra`.

The pairs have to match: `-4` on entry with `+4` on exit keeps the stack balanced.

In pseudocode:

```
food_x = x
food_y = y
set_pixel(x, y, RED)
return
```

Pasted text.txt

TXT

correct ?

Checking which assembler directive spelling MARS accepts ▸

Almost, but it has 3 bugs. The first two would stop it working, and the third would corrupt memory.

1. `read_input` reads the wrong register (the keyboard would never work)

```asm
lw $t1,0($t1)     # wrong: $t1 isn't the keyboard address
```

It must read from `$t0`, which holds `0xFFFF0000`:

```asm
lw $t1,0($t0)
```

2. `place_food` overwrites x with y

```asm
move $s0,$a0 #random y    # wrong: this replaces the x you just saved
```

The second one must go into `$s1`:

```asm
move $s1,$a0
```

Otherwise `$s1` is never set, so the food ends up at a garbage y, and the overlap check compares against that garbage.

3. The wall check lets the snake go one cell too far

```asm
bgt $s2,$t0,us_dead     # x = 64 is not caught
bgt $s3,$t0,us_dead     # y = 32 is not caught
```

Valid columns are 0 to 63 and valid rows are 0 to 31, so 64 and 32 are already out of bounds. Use `bge`:

```asm
bge $s2,$t0,us_dead
bge $s3,$t0,us_dead
```

With `bgt`, a head at y = 32 gets drawn past the end of the display, which overwrites the start of your `snake_x` array. At x = 64 it wraps onto the next row instead.

Smaller things:

- `.global main` should be `.globl main`, which is the spelling MARS documents. It only matters if MARS complains about the directive.
- You print "Game over!" and then the score with no separator, so it shows as `Game over!5`. Printing `msg_nl` between them fixes that. At the moment `msg_nl` is declared but never used.

The rest matches what we went through: `set_pixel`, `clear_screen`, `init_game`, the direction checks, the self-collision loop, the food check, the tail erase, the shift loop and the `pf_chk` loop are all correct. Your commented-out initialization lines in `init_game` are fine too, since `.data` already sets those values.

With those three fixes it should run. If it still misbehaves, tell me what you see on the display.

Claude works directly with your codebase Install